

UNIVERSITY OF BIRMINGHAM

School of Engineering

Computing for Engineers – Assessed Lab 1 (approx. 2 hour exercise)

06-07 November 2023 (Week 7 Semester 1)

This is an assessed lab exercise.

Statement of good academic conduct

By submitting this assignment, I understand that I am agreeing to the following statement of good academic conduct.

I confirm that this assignment is my own work and I have not worked with others in answering the exercises in this assignment.

I confirm that I have not asked, or paid, others to undertake any part of this work for me.

I confirm that I have read and understood the University regulations on plagiarism
<https://intranet.birmingham.ac.uk/as/registry/policy/conduct/plagiarism/index.aspx>

I understand that if concerns are raised about my work I may need to participate in a viva (oral examination) of my work. I also understand that my progress and/or graduation may be delayed whilst these concerns are investigated under the Code of Practice on Academic Integrity or Code of Practice on Plagiarism.

Your C Source code for Exercise A and Exercise B must be uploaded to Canvas by
Tuesday 07 November 2023, 11:59 (midday)

Your code must be uploaded as a **.c** file (C Source file).
In most cases you will need to upload the two files called “main.c” (one for each exercise).

Do not upload .docx files, .pdf or any other format.

Do not email the code to the course lecturer or to any other member of staff or PGTA.
Only Canvas submissions will be marked.

If you have any problems uploading your code, consult the Module Lecturer **well before** the above deadline. Once the deadline has expired, your mark for the Assessment **will be deducted 5 marks per day or part of a day.**

The Module Lecturer cannot extend any deadlines for this assignment for any reason. If you wish to be considered for an extension, you will need to apply for Extenuating Circumstances via Engineering Welfare.

Do not discuss the exercise with any other person, inside or outside the University. Do not get help from any other person. The submitted assessment must be your own work.

You should only use Canvas, your own notes and your own programs as sources of help.

Credit will only be given for techniques which have been discussed during Lectures, Labs or Tutorials. Credit will not be given for techniques learnt or researched elsewhere.

Acceptable sources of help:

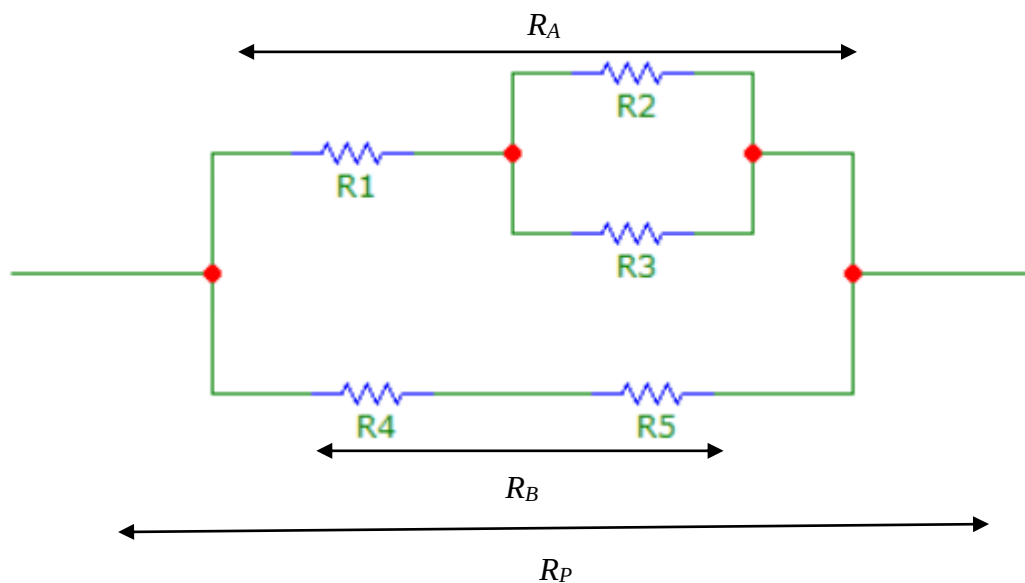
- Your own notes
- The Study Guide on Canvas for this module
- The Lecture / Lab and Tutorial videos for this module on Canvas / Panopto
- Any programs which you have written yourself during the labs this Semester
- You may use published Books, as long as you provide a full reference (Name and Author of Book, Publisher Name, Year of Publication).

Sources which are **not acceptable**:

- Help from any other person, inside or outside the University
- Any website, forum, social media, video, audio, email or messaging, or any other information source, other than the Canvas page for this module
- Direct or indirect usage of any chatbot or AI engine
- Self-published books and course notes from other Universities or online courses

Assessed lab exercises

A. A resistor network has the following layout:



The equations for the above resistances are given by:

$$R_A = R_1 + \frac{1}{\frac{1}{R_2} + \frac{1}{R_3}}$$

$$R_B = R_4 + R_5$$

$$R_P = \frac{1}{\frac{1}{R_A} + \frac{1}{R_B}}$$

A number of identical resistor modules each of resistance R_P can be placed together in series. The resistance R_Q of the whole network is equal to the sum of N resistor modules each of which has resistance R_P .

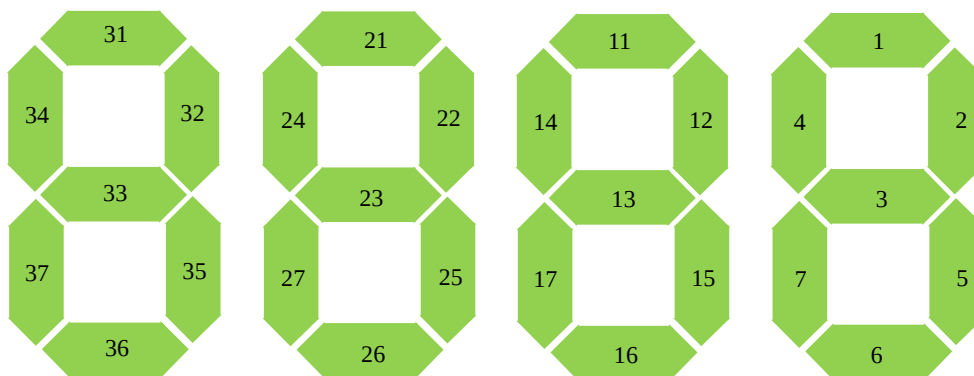
Write a program to enter R_1 , R_2 , R_3 , R_4 , R_5 , and the number of resistor modules N – these should be entered **in this order**. Then **work out and display** R_P and R_Q . You may use your own formula to work out R_Q or you can use a loop, whichever you prefer.

An error message should be displayed if R_2 , R_3 or N is zero or negative, the resistance should not be displayed, and the program should exit.

Marks will be deducted:

- If you introduce any other user interface options not specified in this lab sheet
- If R_1 , R_2 , R_3 , R_4 , R_5 and N are not entered in your program in the **same order** that they are specified on this lab sheet.
- The program **should not ask** for another input if an illegal value of N is entered. If such a value is entered then the program should display the error message and exit.

B. A digital clock is constructed from four *seven-segment LED* displays:



Each numbered segment is part of an electronic circuit, where if the voltage was set to high then that segment will light up. For example, to light up “1111”, we would need to activate the following segments:

2 5
12 15
22 25
32 35

Write a program which does the following:

- Reads in a four digit number, which is the 24-hour clock time
- Prints an error message if the time is illegal, and no segments should be activated
- Extracts each digit, and outputs a list of activated segments for each digit (shown here for the right hand column of the clock):

Digit	Segments to light up
1	2, 5
2	1, 2, 3, 6, 7
3	1, 2, 3, 5, 6

Your program should only output the segment values if the digits are “1”, “2”, or “3”. Any other digits **should be ignored**.

Your program should:

- Loop repeatedly until the user enters “9990”.
- Use a *for* loop to iterate across the four columns (1000, 100, 10, 1)
- For each column, the correct digit (1, 2, 3) should be displayed, and for each digit the correct segments should be printed on the screen.
- Your program should make use of the fact that the segment numbers increase by 10 for each digit, going from right to left (see diagram of clock on previous page)
- A working program should follow the user interface in this specification **exactly**. Otherwise marks will be deducted.

Hint

To extract a digit from a 24 hour clock time, you may find the following formula useful:

$$digit = \frac{c \bmod (10h)}{h}$$

c is the 24 hour clock time, “mod” is the modulus operator, h represents the digit position you want to extract (1000, 100, 10 or 1).

Example output:

```
Enter 24-hour clock time: 2469  
Error - illegal time
```

```
Enter 24-hour clock time: 0978  
Error - illegal time
```

Enter 24-hour clock time:1122
The digits you need to connect are:
Digit 1 needs 32 35
Digit 1 needs 22 25
Digit 2 needs 11 12 13 16 17
Digit 2 needs 1 2 3 6 7

Enter 24-hour clock time:1234
The digits you need to connect are:
Digit 1 needs 32 35
Digit 2 needs 21 22 23 26 27
Digit 3 needs 11 12 13 15 16
Digit 4 ignored

Enter 24-hour clock time:1445
The digits you need to connect are:
Digit 1 needs 32 35
Digit 4 ignored
Digit 4 ignored
Digit 5 ignored

Enter 24-hour clock time:9990
End of the program

END OF THE ASSESSMENT